

RNA-seq Ingestion & Platform

A data-engineering portfolio project

Part 1 built and tested; Parts 2 and 3 designed

Every choice framed as "X over Y, because Z."

1 · The problem, in one line

Four teams. Four numbers. One study.

The pipeline today = **ad-hoc scripts run by hand**. That causes:

- data loaded **by hand** → sometimes twice, sometimes stale
- words like "*evaluable*" and "*QC pass*" live **in people's heads**, not in code
- when a number breaks, **nobody is told**

The fix: land data **once** · define it **once** · make drift **loud**.

2 · What I built vs. designed

Task	What	Status
1 · Ingestion	parse → validate → normalise → load	Built + tested (25 tests)
2 · Gold model	analytics star schema	Designed: tables, history, queries
3 · Platform	target platform + migration	Designed: picks, phases, governance

Scope: Part 1 is implemented; Parts 2 and 3 are design. The roadmap is on the last slide.

3 • Part 1: how the pipeline is shaped

```
JSON → parse → validate → normalise → load → Postgres
      |_____ pure code, no database _____|         | one transaction |
```

Plain version: the "brain" (read, check, tidy) needs **no database to test**, so tests are fast, and that brain survives the move to a new platform.

- only the last step (`load`) touches the database
- 17 of the 25 tests run with **no Docker** at all

4 · Schema evolution: the two-box design

Vendor fields **keep changing** (new assays, a 2nd lab). Goal: store them **without a migration every time**.

Promote → real columns	Tail → one JSON column
the ~10 fields we filter/constrain	everything else, incl. unknown fields
<code>sequencing_batch</code> , <code>cro</code> , <code>gene_model</code> ...	<code>library_prep</code> , <code>flow_cell_id</code> ...
constraints + indexes	flexible, still searchable

- new field arrives → lands in the JSON box, **zero migrations**
- the **raw payload is kept whole** (audit) · only *added* columns → **old dashboards unbroken**

5 · Idempotency: re-runs are safe

Re-sending the same sample must never duplicate or corrupt it.

Chosen: keep one current row per sample + keep every version in an audit log.

- **Rejected** *skip duplicates*: would keep **stale** numbers
- **Rejected** *store every version as a row*: breaks "one row per sample" and the **UNIQUE** rule
- **Chosen** *update-in-place, if newer*: plus full history on the side

Same delivery arrives again → **do nothing** (retries are safe).

A newer re-processing → **update, and log what changed.**

6 · Robustness: bad data fails loudly, writes nothing

A broken payload → all errors reported at once, nothing written:

```
{ "status": "rejected", "error_count": 6, "errors": [  
  {"path": "sample.qc_metrics.total_reads", "value": "52381204",  
    "message": "Input should be a valid integer"}, ... ]}
```

- **strict types:** the text `"52381204"` is *rejected*, not silently turned into a number
- every error names its **exact location** in the JSON
- **all-or-nothing:** the bad one goes to a "dead-letter" log, the database is untouched

7 · Tests & outcomes: 25 green

Fixture	What it proves
RNA-BULK-006	happy path: 1 sample + 15 genes
RNA-BULK-007	2nd lab's field names auto-mapped
RNA-SC-008	single-cell: no TPM, cell metrics, 401M reads
...006_duplicate	re-run updates in place · replay = no-op
...009_malformed	6 errors, database unchanged

End-to-end: 5 seed rows → **8 samples, 60 gene rows, 1 rejected**; old view intact.

8 Part 2: the analytics model (star)

```
dim_subject    dim_gene    dim_batch
(history)      (by gene version) (lab/run)
  \            |            /
  FCT_GENE_EXPRESSION (one row per sample x gene)
  /            \
dim_sample      dim_date
  \            /
  FCT_SAMPLE_QC (one row per sample)
```

Star schema: scientists can read it without a data engineer. Two fact tables at **different levels of detail** keep the huge expression table lean.

9 Â• Part 2: handling changes over time

- **Diagnosis corrected** → keep history: a cohort built in March is still rebuildable in June.
- **Gene labels revised** → *not* a normal history case. Expression only compares **within one gene-model version**, so each version is its **own set of rows**, never overwritten.
- **A typo fix** → just overwrite; it doesn't deserve a new record.

10 Â• Part 2: two example questions

- (a) which genes differ **disease vs control**, for one indication
- (b) is **quality drifting** by lab/batch over time?

Honest caveat: (a) is a **screening** view, not the final statistics. Real differential-expression tools (DESeq2/limma) do that. The platform's job is to hand them a **clean, batch-labelled matrix**, not to replace them.

11 Â· Part 3: the target platform

```
Sources → Landing → Bronze → Silver → Gold → Marts → BI
(API,LIMS, (raw,      (raw+   (clean) (star) (wide) (Power BI /
.h5)      immutable) log)                notebooks)
```

Lakehouse: Databricks or MS Fabric

Gold → (reverse-ETL) → legacy Postgres → old dashboards stay alive

Nextflow / nf-core: triggered, not rebuilt

Layers: raw → clean → analytics. Push a copy back to the old database so **nothing breaks during the move.**

12 Â• Part 3: picks & trade-offs

Layer	Pick	Instead of...
Lakehouse	Databricks / MS Fabric	Snowflake (if it were BI-first, no ML)
Orchestration	Dagster	native pipelines / Airflow
Bio workflows	Keep Nextflow	rebuilding it (classic mistake)
Transform	dbt	hand-rolled SQL
Ingestion	buy connectors, build the vendor API	build everything

Databricks vs Fabric: Databricks for multi-cloud + heavy ML; **Fabric** for a Microsoft/Power BI shop wanting one low-ops tool.

13 · Migration + what I'm *not* doing

Move piece by piece (strangler-fig), 4 phases:

foundations → take over the painful ingestion → build Gold + **reconcile old vs new numbers** → retire scripts one owner at a time.

Not in year 1: raw sequence-file lake · ML platform · full GxP validation · streaming.

Roadmap: real matrix files · CI + secrets · tissue vocabulary · dbt build of the star.

■ **GxP-ready, not GxP-now.** Version control, immutable raw data, lineage, tests: the groundwork, for free, today.